COAL

Lecture 7

Subroutines/Procedures

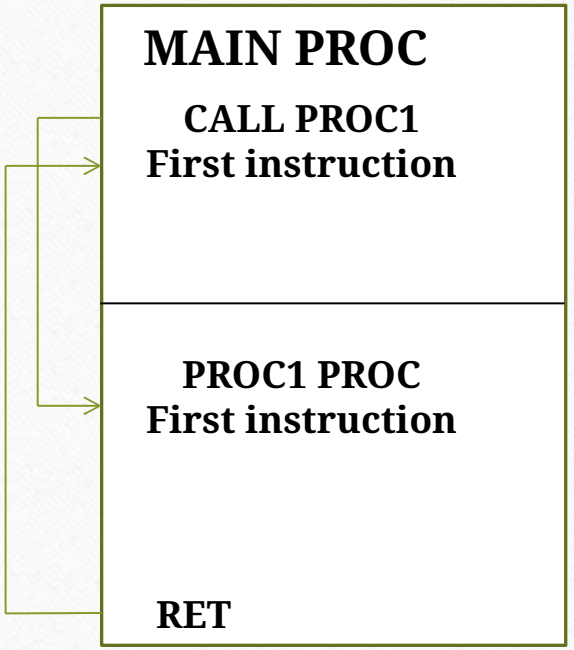
- Used almost interchangeably with Procedures
- Block of code designed to perform a specific task that can be reused multiple times within a program
- Allow for modular programming, making code more organized and easier to manage
- Allows you to avoid repeating code by allowing common functionality to be called from different places within the program.
- When a subroutine is called, control is transferred to the subroutine, and after it finishes, control is returned to the calling code.

Calling a Subroutine

- A procedure is the assembly equivalent of a Java or C function
- **CALL** instruction is used to call a subroutine.
- When a subroutine is called, the return address is pushed onto the stack, so after the subroutine finishes, the program can jump back to where it was called from.
- **RET** instruction is used to return control to the calling function.

```
sample PROC
    .
    .
   ret
sample ENDP
```

Procedure Call and Return



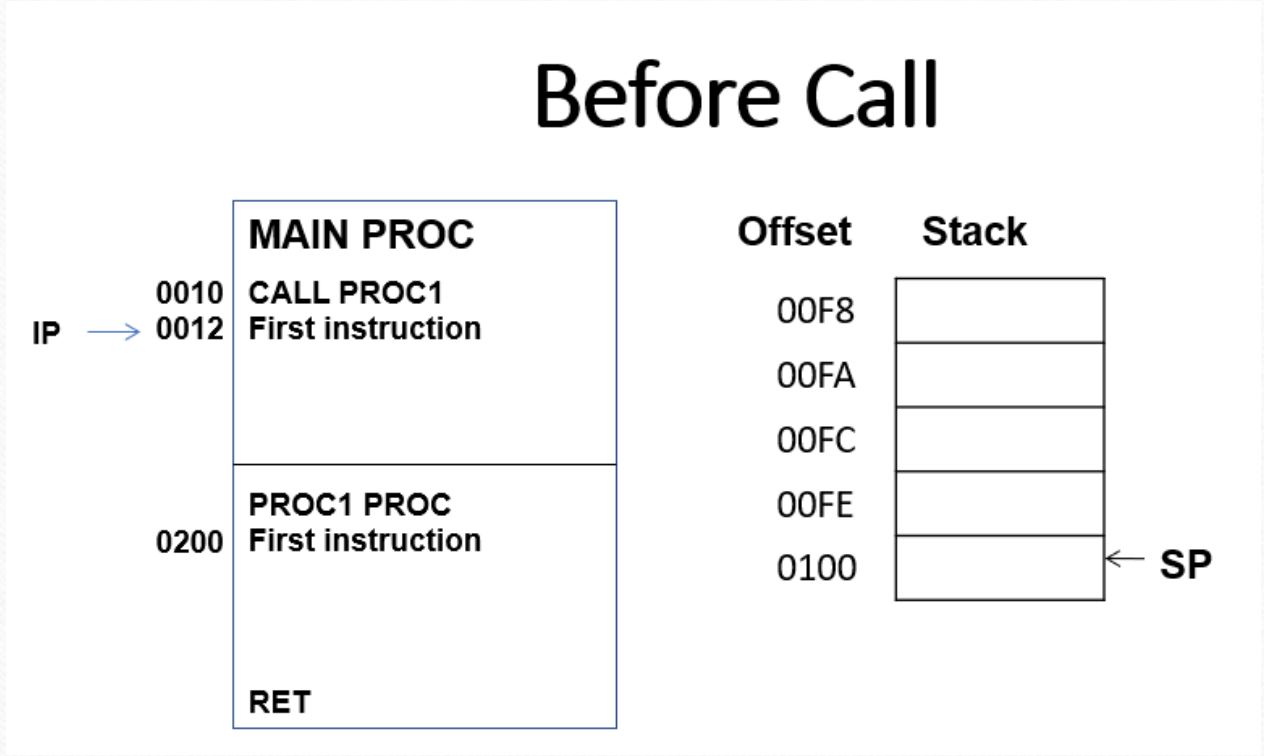
CALL and RET Instructions

- The CALL instruction calls a procedure
  - Pushes offset of next instruction on the stack
  - Copies the address of the called procedure into IP (Note: IP=Instruction Pointer)
- The RET instruction returns from a procedure
  - Pops top of stack into IP

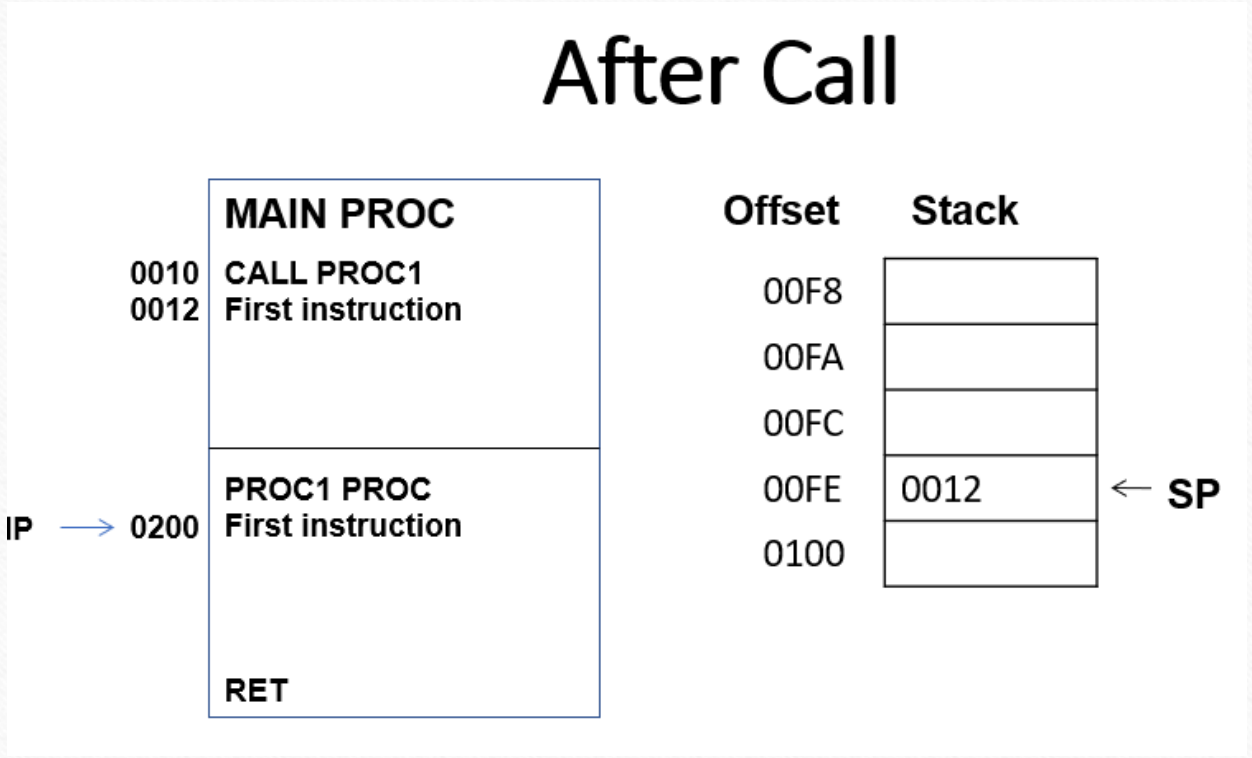
CALL and RET Instructions

Instruction	Operation	Description
Call label/ offset (2 bytes)	Push IP IP ← Label	Calling procedure that is defined in the current code segment
Call label/ segment: offset (4 bytes)	Push CS Push IP IP ← offset CS← segment	Calling procedure that is defined outside the current code segment
Ret	Pop IP	Returns control from procedure defined in the current code segment.
Retf	POP IP POP CS	Returns control from procedure defined outside the current code segment.

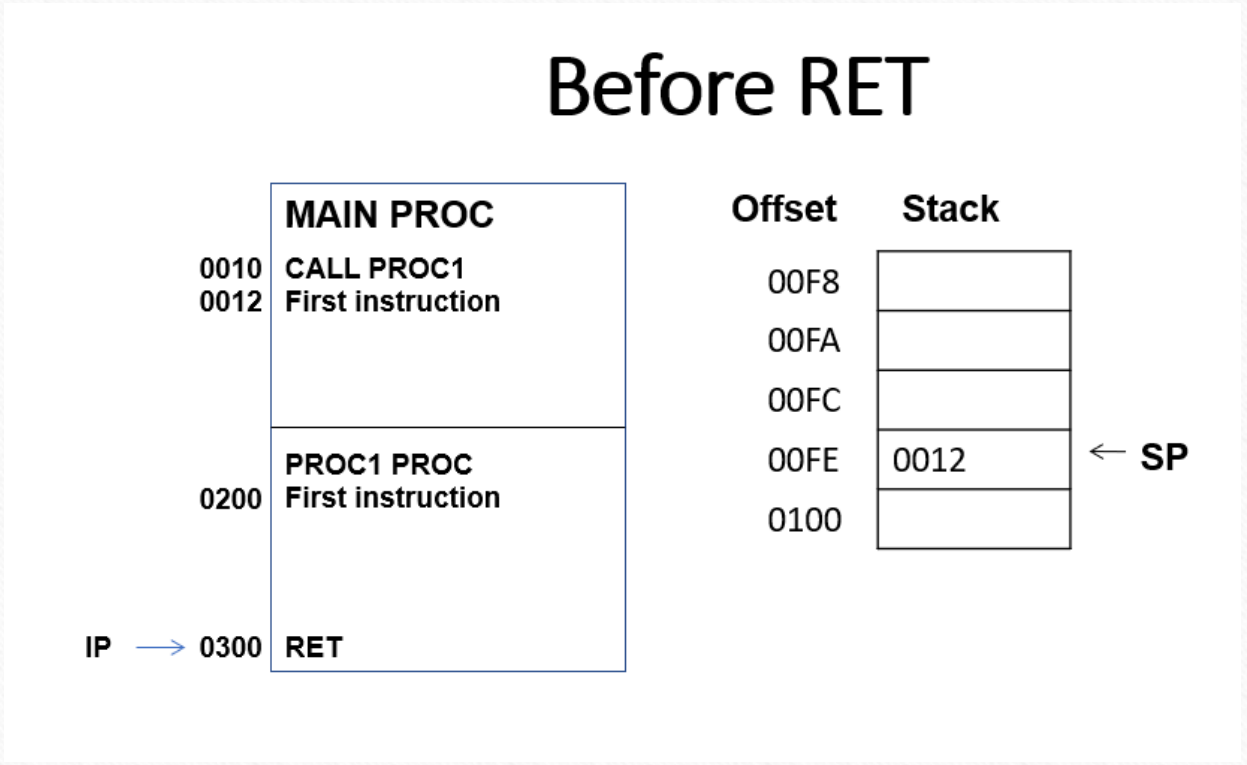
Stack Representation



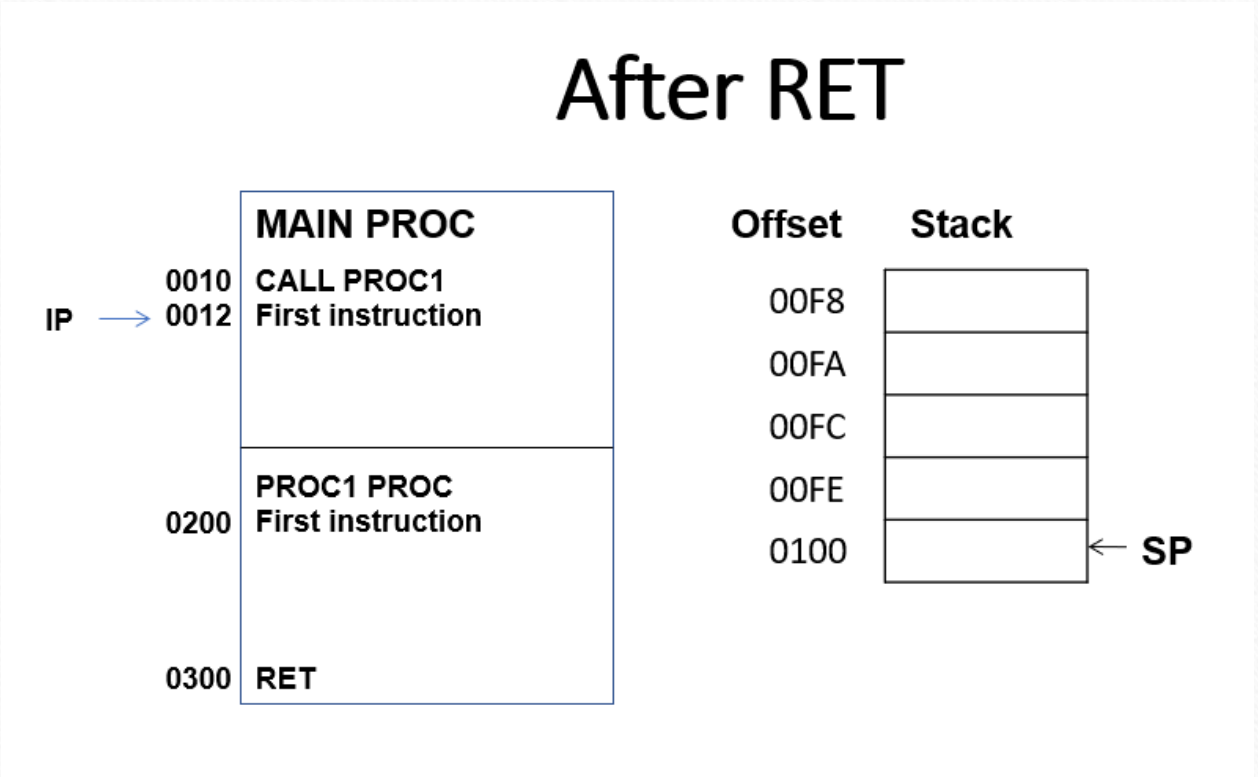
Stack Representation



Stack Representation



Stack Representation



Main Procedure

- Called by operating system

```
.model small

.stack

.data

.code
Main proc

Mov ax,@data
Mov ds,ax

.exit
Main endp
```

Addition Procedure

```
.model small

.stack

.data

.code
Main proc                ;Defining main procedure
Mov ax,@data
Mov ds,ax

Call addition            ;Calling procedure
.exit
Main endp                ;main procedure ends here

Addition proc            ;defining procedure
Add ax,bx                ;adding two registers

Ret                      ;return control back to calling procedure
Addition endp
```

Stack Management in Subroutines

- Subroutines typically use the stack to save and restore registers.
- This ensures that the calling program’s state is preserved across the subroutine call.
- The CALL instruction automatically pushes the return address onto the stack.
- The RET instruction pops the return address from the stack and jumps to it.

Use of Stack for Parameters

- Temporary storage during function execution.
- Allows multiple nested calls without conflict.
- Efficient parameter handling in assembly.

Steps for Parameter Passing

- **Push Parameters**: Caller pushes arguments onto the stack.
- **CALL Instruction**: Control jumps to the subroutine.
- **Access Parameters**: Subroutine accesses parameters via the stack.
- **Perform Operations**: Subroutine uses parameters to execute tasks.
- **Clean Up**: Caller removes parameters after execution.

Addition Proc with Stack

```
.model small
.stack
.data
.code
Main proc
Mov ax,@data
Mov ds,ax
Mov ax,5
Mov bx,10
Push ax          ; pushing value of ax, which is 5 to stack
Push bx          ; pushing value of bx, which is 10 to stack
Call addition    ; calling procedure
Pop bx           ; removing number 10 from stack
Pop ax           ; removing number 5 from stack
.exit
Main endp
Addition proc    ; defining procedure
Push bp          ; storing value of BP on stack so that we can restore it later
Mov bp,sp        ; to access stack without pop instructionùmoving TOS to bp
Mov ax,0         ; no need to push AX as it is safe to use AX register
Add ax,[bp+4]    ; [bp+4] contains value 10
Add ax,[bp+6]    ; [bp+6] contains value 5
Pop bp           ; restoring value of BP from stack
Ret              ; transferring control back to the calling procedure
Addition endp
```

Passing Array in Proc

```
.model small
.data
array db 1,2,3,4,5
.code
Main proc
Mov ax,@data
Mov ds,ax
Mov bx,offset array      ; base address of array
mov ax,5                 ; size of array
Push bx
Push ax
Call addition            ; calling procedure
pop bx                   ; removing parameters from stack
pop bx                   ; removing parameters from stack
.exit
Main endp
Addition proc            ; Defining procedure
push bp                  ; saving the value of BP on stack before using it
push cx                  ; saving the value of CX on stack before using it
push si                  ; saving the value of SI on stack before using it
mov bp,sp                ; moving top of the stack to bp
mov cx,[bp+8]            ; reading size of array
mov ax,[bp+10]           ; reading base address of array
mov bp,ax                ; moving base address to BP
mov ax,0                 ; it is safe to use ax without pushing it
mov si,0                 ; assigning index register with 0
l1:
Add al,ds:[bp+si]        ; accessing value 10
inc si                   ; incrementing index by 1 as the array is of a byte type
loop l1
pop si                   ; Restoring original value of SI from stack
pop cx                   ; Restoring original value of CX from stack
pop bp                   ; Restoring original value of BP from stack
ret                      ; returning control back to calling procedure
Addition endp
```

Creating Local Variables

```
.model small
.data
.code
Main proc
 push bp                        ; storing value of bp on stack
mov bp,sp
sub sp,4                        ;creating space on stack for two variables
 mov word ptr [bp-2],5          ; assigning value to local variable
 mov word ptr [bp-4],10         ; assigning value to local variable
                                ; now you may PUSH and POP equal number of times here
mov sp,bp                       ;destroying local variables before returning
pop bp                          ; restoring value of bp from stack
ret
.exit
Main endp
```

Methods of Returning Values

- **Via Registers**: Common method, uses AX, BX, etc.
- **Via Stack**: Push result onto the stack before RET.
- **Direct Memory**: Store result in a memory location.

Returning Values via Registers

- **Example**: AX register
- SUB:
  - MOV AX, 10 ; Return value in AX
- RET
- Fast and simple for single values.

Returning Values via Stack

- **Steps**:
  - Push result onto the stack in subroutine.
  - Caller pops the result after RET.

```
SUB:
  PUSH AX     ; Push result
  RET

; Caller
CALL SUB
POP BX       ; Get result from stack
```

Returning Values via Memory

- Useful for large or complex data.
- Subroutine writes directly to a pre-defined memory address.

```
SUB:
  MOV [RESULT], AX  ; Store result in memory
  RET
```

Comparison of Methods

Method	Use Case	Advantages	Limitations
Registers	Simple values	Fast and efficient	Limited capacity
Stack	Nested calls	Flexible, supports recursion	Slower than registers
Memory	Large/complex data	No size limits	Slower due to memory access